From C to Rocq

How a fast sorting routine is proved correct

Laurent Théry

INRIA, Stamp Team
Laurent.Thery@inria.fr

Abstract. djbsort is a small C library that sorts arrays of 32-bit integers. It is fast, and it is careful: the comparisons it makes never depend on the values it is sorting. This note explains, step by step, how its two implementations — the portable one and the one written with AVX2 vector instructions — have been proved to sort, in the Rocq prover. It is written for readers who know neither sorting networks nor Rocq. Every idea is introduced with a small example, and the real code is shown only after the idea is clear.

1 The programs, and what makes them special

A word first on the tool. Rocq — until recently called Coq — is a proof assistant. One writes definitions and statements in it, and then a proof, and the machine checks every step. Nothing is believed because it looks right. When a proof is finished, one can also ask the machine which assumptions it rests on; the answer for the proofs described here is "none".

djbsort sorts an array. What it never does is ask "is this value smaller than that one?" and then take one road or the other. Instead it uses a single small step, applied to a fixed pair of places in the array:

```
#define int32_MINMAX(a,b) \
do { \
  int32 ab = b ^ a;  int32 c = b - a; \
  c ^= ab & (c ^ b); c >>= 31; c &= ab; \
  a ^= c; b ^= c;    \
} while(0)
```

The details do not matter here. What matters is the effect: after the step, `a` holds the smaller of the two values and `b` the larger. There is no `if`. The processor performs the same instructions whatever the data, which is what makes the routine constant-time, and useful in cryptography.

We call one such step a **comparison**, and write it as a pair of places, for instance $(3, 5)$: compare what is in place 3 with what is in place 5, and put the smaller one in place 3.

Because there is no branching on data, the **sequence of pairs** the program uses is decided in advance. It depends on the length of the array, and on nothing else. A program of this shape is called a **sorting network**, and the whole verification rests on that one observation.

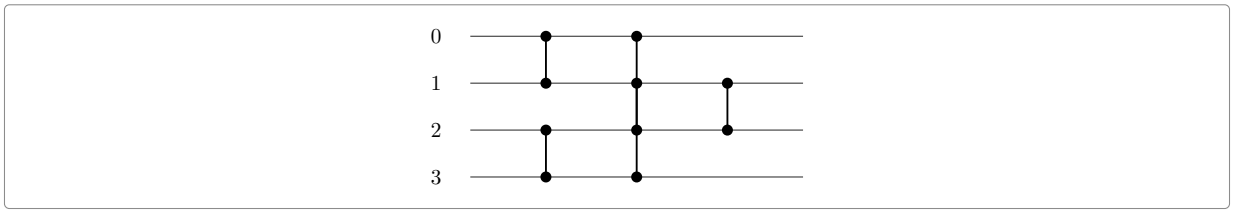

Figure 1: A network on four places. Time runs left to right. Each vertical bar is one comparison: the smaller value goes to the upper end, the larger to the lower one. This one sorts any four values.

It is worth running one input through Figure 1 by hand, because everything later is about such lists of pairs. The network is $(0, 1), (2, 3), (0, 2), (1, 3), (1, 2)$, and we start from $3, 1, 4, 2$:

comparison	array	what happened
–	3 1 4 2	the input
$(0, 1)$	1 3 4 2	3 and 1 were in the wrong order
$(2, 3)$	1 3 2 4	4 and 2 were in the wrong order
$(0, 2)$	1 3 2 4	1 is already the smaller
$(1, 3)$	1 3 2 4	3 is already the smaller
$(1, 2)$	1 2 3 4	the last two settle

2 Why one can check a network with zeros and ones

Suppose someone hands you a network on four places, like Figure 1, and claims it sorts. How would you check it? Trying every possible input is hopeless: there are far too many arrays of four 32-bit integers.

There is a classical way out, and it is the first idea of the whole development.

> **The zero-one principle.** If a network sorts every array made only of zeros and ones, then it sorts every array of numbers.

Here is why, in one paragraph. Take any array that the network fails to sort, and look at the first place where the output is too large: some value v ends up above some value u with $u < v$. Now replace, in the original input, every value below v by 0 and every value from v upwards by 1, and run the network again. A comparison behaves in exactly the same way as before — the smaller of two values stays the smaller after the replacement — so the zeros and ones travel along the same wires as the values they came from. Hence the output has a 1 where v was and a 0 where u was, in the wrong order. So a network that sorts all arrays of zeros and ones cannot fail.

For four places this brings the check down from "all arrays of integers" to sixteen cases, which one can do by hand. For a real array — a thousand places, say — there are still 2^{1000} cases, so one does not check them but proves the statement. Only the very small pieces, such as the fixed batches of five or twelve comparisons the AVX2 code contains, are settled by letting the machine try all cases.

3 Networks in Rocq

Before going further we have to say how a network is written down in Rocq, because two ways of writing it are used, and the difference matters later.

The array. An array of m values is a `m.-tuple A`: exactly m values, of some type `A` on which there is an order. `A` can be the integers, or the booleans of the previous section; nothing in the development depends on which. The places are numbered by `'I_m`, the whole numbers below m, so an array of four values has places 0, 1, 2 and 3.

The first way: a list of pairs. This is what section 1 gave us, and it needs no machinery: `[:: (0,1); (2,3); (0,2); (1,3); (1,2)]` is Figure 1. The function `pnet` turns such a list into a network, and pairs whose places fall outside the array are simply dropped.

The second way: a list of stages. Hardware does not do one comparison at a time. A vector instruction does eight at once, and a picture like Figure 1 is naturally read in columns: at each step, several disjoint pairs are compared together. One such column is called a **connector**, and it is the type the mathematical side is built from:

```
Record connector (m : nat) := connector_of {
  clink  : {ffun 'I_m -> 'I_m};
  cflip  : {ffun 'I_m -> bool};
  cfinv  : [forall i, clink (clink i) == i];
  cflipinv : [forall i, cflip (clink i) == cflip i] }.
```

Word by word:

- `clink` says, for each place, which place it is paired with. A place that is paired with itself is left alone by this stage.
- `cflip` says, for each place, which way round its pair goes: `false` for "smaller value to the smaller place", `true` for the other way.
- `cfinv` is a condition, not data: following `clink` twice comes back to where you started. It says the pairing really is a set of disjoint pairs.
- `cflipinv` is the other condition: the two ends of a pair agree on the direction. It would make no sense for one end to want the smaller value and the other end the larger.

Running one stage is then what you would expect: each place looks at its partner and keeps the min or the max.

```
Definition cfun c t :=
  [tuple let min := min (tnth t i) (tnth t (clink c i)) in
         let max := max (tnth t i) (tnth t (clink c i)) in
         if i <= clink c i
         then if cflip c i then max else min
         else if cflip c i then min else max | i < m].
```

Read it as: for every place `i`, take the two values `t i` and `t (clink c i)`; if `i` is the smaller of the two places, keep the min, unless the flip says otherwise; if it is the larger, keep the max, unless the flip says otherwise. Both ends of a pair use the same line of code, and they agree because of `cflipinv`.

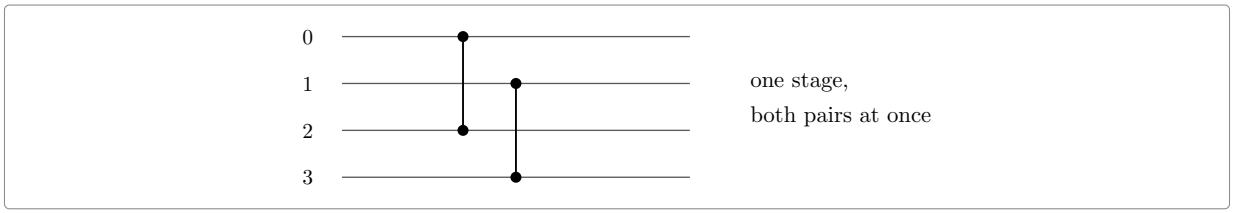

Figure 2: One connector on four places: `clink` pairs 0 with 2 and 1 with 3, and `cflip` is false everywhere. The two comparisons touch four different places, so the machine may do them together.

A **network** is then a list of stages, and running it means running them one after the other:

```
Definition network := seq (connector m).
Definition nfun n t := foldl (fun t c => cfun c t) t n.
```

The two ways of writing a network fit together: a single pair (i, j) is a stage that touches only two places (`cswap i j`), so a list of pairs is a network in which every stage does one comparison. That is exactly what `pnet` builds. The program side of the proof uses lists of pairs, because that is what a program performs; the mathematical side uses stages, because that is what one can do induction on.

Now the property of interest can be written down, and it is the zero-one principle of the previous section, taken as the definition:

```
Definition sorting :=
  [qualify n | [forall r : m.-tuple bool, sorted <=%O (nfun n r)]].
```

- `m.-tuple bool` is an array of exactly `m` booleans, so `r` ranges over arrays of zeros and ones;
- `nfun n r` runs the network `n` on the array `r`;
- `sorted <=%O` says the result is in increasing order;
- `[forall r ...]` says this holds for every such array;
- `n \is sorting` is then read "n is a sorting network".

Because of the zero-one principle this single line is as strong as sorting arrays of integers, and the library proves that once, for any network.

3.1 Why the mathematical networks sort

The AVX2 code follows Batcher's **bitonic** sorter, and it is worth seeing why that works, since the whole right-hand side of the proof rests on it.

A sequence is **bitonic** if it goes up and then down — 1, 4, 7, 6, 2 — or becomes such a sequence if you rotate it. Two sorted runs, one up and one down, always give a bitonic sequence when put end to end.

Now take a bitonic sequence of $2m$ values and compare each place i with the place $i + m$, keeping the smaller on the left. This one stage is the **half-cleaner**. After it, three things are true, and they are the heart of the matter:

1. every value in the left half is at most every value in the right half;
2. the left half is still bitonic;
3. the right half is still bitonic.

So the problem falls apart: no value ever has to cross the middle again, and each half is a smaller copy of the same problem. Repeating the half-cleaner on halves, then on quarters, and so on, sorts the whole thing. That is `half_cleaner_rec`.

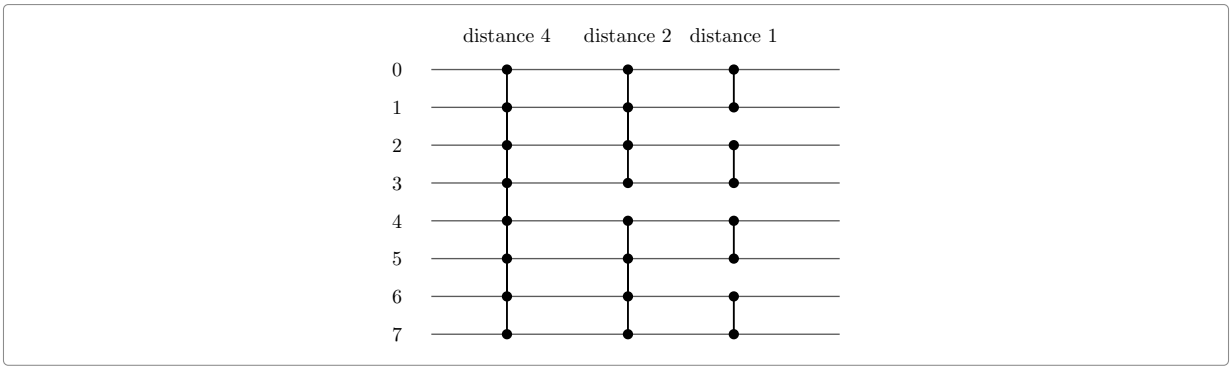

Figure 3: Sorting a bitonic sequence of eight values: one half-cleaner at distance four, then one in each half at distance two, then one in each quarter. After the first column, nothing crosses the middle again.

A full sorter follows: sort the first half upwards, sort the second half downwards, put them together — the result is bitonic — and then clean it with Figure 3. That is a recursion, and it is the network the AVX2 code implements.

The portable code follows a different plan, Knuth's merge exchange, but the shape of the argument is the same: a network built by a recursion, proved to sort by induction, with the zero-one principle doing the work at the bottom.

4 The shape of the proof

There are two objects, and they are not the same kind of thing.

On one side there is a **network** that mathematics knows how to reason about, and that can be proved to sort by induction: Batcher's bitonic sorter for the AVX2 code, Knuth's merge exchange for the portable code. On the other side there is a **program**: loops, vector registers, shuffles, masks.

The proof therefore has three parts: the network, the program, and a bridge saying that the program performs the comparisons of the network.

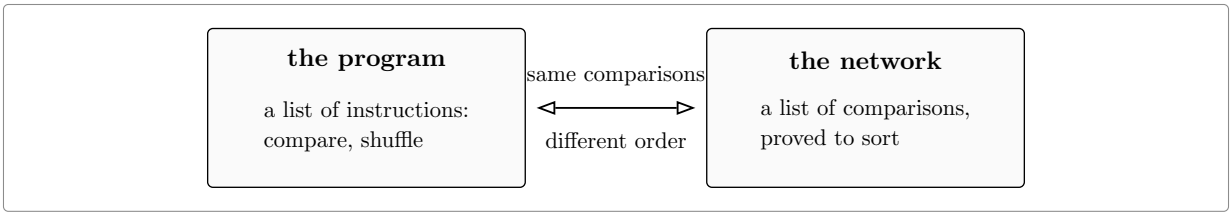

Figure 4: The two sides and the bridge between them. The left box is what the machine runs; the right box is what one can reason about; the bridge says they perform the same comparisons.

The left box is built once and for all as a small language. The right box is classical mathematics. Almost all the work is in the bridge, and the rest of this note is about the five techniques it needs.

5 Technique one: writing the program down

A program is written as a list of instructions. There are only three of them:

```
Inductive item : Type :=
  | Cmp   of nat * nat            (* one compare-exchange *)
  | Vcmp  of seq (nat * nat)      (* one vector compare-exchange: its lanes *)
  | Vshuf of cperm m.             (* one vector lane shuffle *)
```

- `Cmp (3, 5)` is the scalar step of section 1: compare places 3 and 5.
- `Vcmp` is the same thing done by the vector unit. One AVX2 instruction compares eight pairs at once, so the instruction carries the list of the eight pairs it performs.
- `Vshuf` moves values about without comparing anything. This is what a lane shuffle or a transpose does.

A list of such instructions is a `prog`. There are then two ways to read a program, and the difference between them is the heart of the matter.

Running it. `pfun p t` takes an array `t` and returns the array after the program has run. This is what the machine does.

Reading it. `pflat p` walks through the program and collects the pairs it compares, ignoring the shuffles. That gives a plain list of comparisons — exactly the right-hand box of Figure 4.

The catch is that a shuffle changes what "place 3" means. Suppose a program first exchanges the values in places 1 and 2, and then compares places 0 and 1. In terms of the values, the comparison is between the value that started in place 0 and the one that started in place 2.

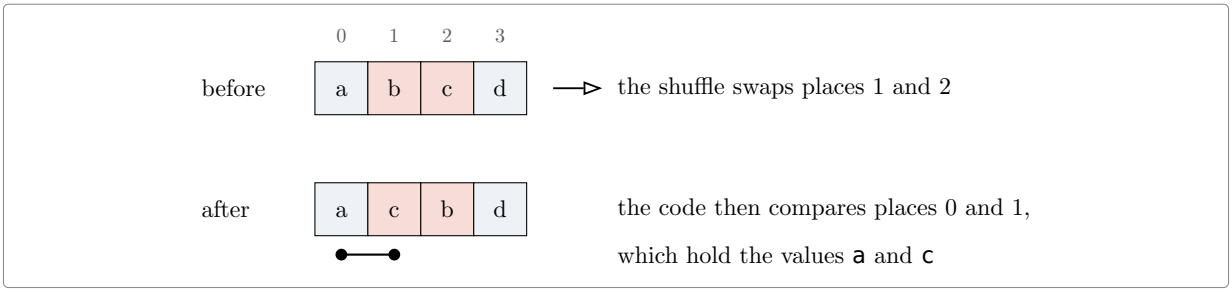

Figure 5: A shuffle changes the meaning of a place. The comparison that the code writes as $(0, 1)$ is, in terms of values, the pair (a, c).

So `pflat` does not record the pair the code writes down. It records the pair **renamed by all the moves made so far**. In the example it records $(0, 2)$, because the value called `c` began life in place 2. Every comparison is then named in one fixed way, and the list can be compared with a network.

There is one more twist, and it is the reason the AVX2 proof is arranged the way it is. The program ends with a big shuffle: the values come out of the vector registers in an order that has to be undone.

Rather than track that at every step, all comparisons are named by **the place the value ends up in** when the program finishes. Nothing is lost: it is a change of names, applied everywhere.

6 Technique two: the same comparisons in a different order

Now both sides are lists of comparisons, and one would like them to be equal. They are not, and they cannot be.

The network wants to work **distance by distance**: compare everything a thousand apart, then everything five hundred apart, and so on. The program cannot afford that. A vector instruction handles eight pairs at once, and it is only worth issuing if all eight are ready. So the code works **region by region**: it takes a block of the array, does every distance inside that block while the values are still in registers, and only then moves on.

Both orders contain the same comparisons. The question is whether the order matters.

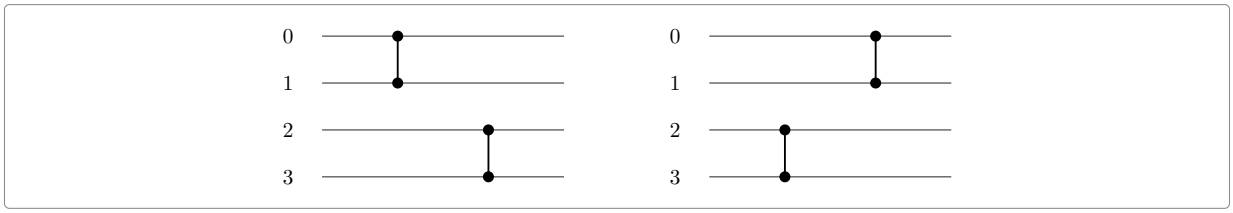

Figure 6: The same two comparisons in the two possible orders. They touch four different places, so nothing can change: whichever is done first, the result is the same.

That is the whole idea, and it is worth stating plainly.

> **Two comparisons that share no place may be swapped.** Doing $(0, 1)$ then $(2, 3)$ gives the same array as doing $(2, 3)$ then $(0, 1)$.

In Rocq, "sharing no place" is a small test on two pairs, and one swap is one step of a relation between lists:

```
Definition dpair (ab cd : nat * nat) : bool :=
  [&& ab.1 != cd.1, ab.1 != cd.2, ab.2 != cd.1 & ab.2 != cd.2].

Inductive dswap (n : nat) : seq (nat * nat) -> seq (nat * nat) -> Prop :=
  dswap_step ps ab cd qs of
    bnd n ab & bnd n cd & dpair ab cd :
      dswap n (ps ++ ab :: cd :: qs) (ps ++ cd :: ab :: qs).

Inductive dequiv (n : nat) : seq (nat * nat) -> seq (nat * nat) -> Prop :=
| dequiv_refl l : dequiv n l l
| dequiv_step l1 l2 l3 of dswap n l1 l2 & dequiv n l2 l3 : dequiv n l1 l3.
```

- `dpair ab cd` says the two pairs have no place in common.
- `dswap n l1 l2` says: the two lists are the same except that somewhere in the middle two neighbouring comparisons, which share no place, have changed order. `bnd n` only says that the places are inside the array.
- `dequiv n l1 l2` says: one list can be turned into the other by a run of such swaps.

And the payoff, proved once:

```
Lemma nfun_dequiv n (l1 l2 : seq (nat * nat)) (t : n.-tuple A) :
  dequiv n l1 l2 -> nfun (pnet n l1) t = nfun (pnet n l2) t.
```

In words: lists related by `dequiv` compute the same function. So if the program's list is a reordering of the network's list, and the network sorts, then the program sorts.

6.1 Doing it without moving one comparison at a time

Swapping neighbours is fine on paper, but the two orders differ by millions of swaps. Writing them out is not an option. What is needed is a way to justify a whole rearrangement at once, and here it is.

Give every comparison a **colour**, with two conditions:

1. two comparisons of different colours never share a place;
2. both lists, read from left to right, contain each colour in the same order.

Then the two lists are related by `dequiv`. The reason is short: a comparison only ever has to move past comparisons of another colour, and those it may move past freely, by the rule above.

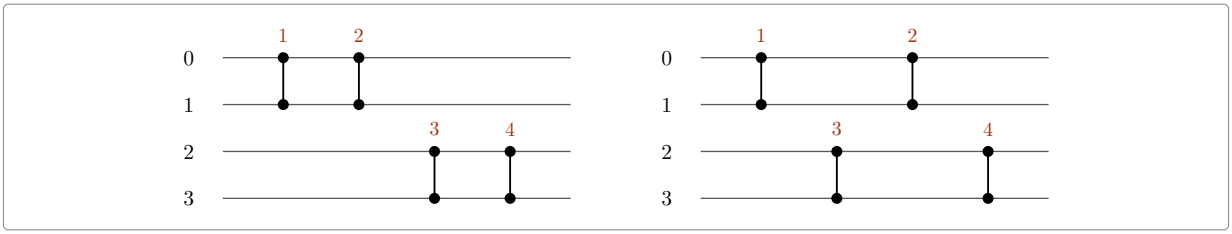

Figure 7: Colour by the half of the array a comparison lives in. The left listing does the top half first; the right listing alternates. Different colours never share a place, and each colour is read in the same order (1 then 2, 3 then 4), so the two compute the same thing.

This is exactly the situation of Figure 4: the program goes region by region, the network goes distance by distance, and the colour of a comparison is the region it belongs to. In Rocq the statement is

```
Lemma dequiv_colour n (c : nat * nat -> nat) (m : nat) (l1 l2 : seq (nat * nat)) :
  ... (* both lists are in range, and every colour is below m *)
  (forall g, g < m -> [seq ab <- l1 | c ab == g] = [seq ab <- l2 | c ab == g]) ->
  dequiv n l1 l2.
```

where `c` is the colouring, and the last line is condition 2: filtering either list by a colour gives the same list. This single lemma settles the reordering in both programs. Only the colouring changes: for the portable code it is the position a chain of comparisons starts at; for the AVX2 code it is the group of eight, or of sixty-four, that a value belongs to.

7 Technique three: sorting downwards without any downward code

Networks like the bitonic sorter do not only sort upwards. Half of the work is sorting a block **downwards**, so that the two halves together can be merged.

The code contains no downward comparison. It does something cleverer: before a block is sorted downwards, every value in it is complemented — all bits flipped, which reverses the order — then the block is sorted upwards as usual, and complemented back at the end. Complementing twice does nothing, so the complements of neighbouring stages cancel, and in the real code the flips are folded into a running **mask**: a pattern that says, for each place, whether the value sitting there is currently complemented.

Here it is on two values. Sorting $(3, 7)$ downwards should give $(7, 3)$. Complement the two values, which turns 3 into -4 and 7 into -8: the pair is $(-4, -8)$. Sort it upwards: $(-8, -4)$. Complement back: $(7, 3)$, which is what was wanted. The sort itself never knew about the direction.

The proof therefore has to carry that pattern. It is written as a predicate:

```
Definition dflP (K : nat) (fl : flips) : Prop :=
  size fl = n /\ forall i, i < n -> nth false fl i = dfl K i.
```

which reads: the mask `fl` has one bit per place, and its bit at place `i` is the bit the merge of size `K` asks for. The proof then shows that each pass of the program keeps this invariant: a shuffle carries the mask with the data, a mask instruction exclusive-ors a pattern into it, and after the last merge the mask is all false — nothing is complemented, which is what "sorted upwards" requires.

8 Technique four: a loop nest is a recursion

Textbooks write the bitonic sorter recursively: sort the two halves in opposite directions, then merge. The code does no such thing. It runs a loop nest, and its innermost line is the one everybody recognises:

```
for (k = 2; k <= n; k *= 2)
  for (j = k/2; j >= 1; j /= 2)
    for (i = 0; i < n; i++)
      if ((i & j) == 0)
        compare(i, i ^ j, ascending: (i & k) == 0);
```

Two whole numbers decide everything: the partner of place `i` is `i` with one bit turned over, and the direction is another bit of `i`. Nothing else.

Are the loop nest and the recursion the same network? Yes, and this is now a theorem rather than a belief. Take eight places. The loop nest runs, in order:

round	distances	what it achieves	in the recursion
k = 2	1	pairs sorted, alternately up and down	the two halves of a half
k = 4	2, 1	runs of four, alternately up and down	the two halves
k = 8	4, 2, 1	the whole array sorted	the final merge

The first two rounds are the two halves of the array being sorted in opposite directions — which is what the recursion asks for — and the last round is the merge. The bit `i & k` is what makes the second half go the other way. In Rocq this is proved as an equality of networks, stage by stage, flips included:

```
Theorem isort_pbsort k : isort k = pbsort false k.
```

`isort` is the loop nest, written down as a network; `pbsort` is the recursive sorter, proved to sort by induction. Before this theorem, the correspondence was checked by running both and comparing traces, for small sizes only.

9 Technique five: reading the loops of the portable code

The portable code is Knuth's merge exchange, and it has the same difficulty in a different dress. Its inner loop walks a **chain**: it takes one place, and compares it with places further and further away, halving the distance each time. The network wants the opposite grouping: all the longest comparisons first, then all the next longest, and so on.

So the two lists are the same comparisons in a different order once more, and the same colouring machinery applies. Two facts do all the work:

- a doubled sweep and the alternating one differ only by comparisons on even places against comparisons on odd places, which never share a place;
- pulling the longest comparison out of every chain is legitimate, because a later chain's long comparison shares no place with an earlier chain's short ones.

Both are instances of the lemmas of section 6, and both used to be proved from scratch, at the level of functions rather than lists. The programs are quite different; the argument is the same one twice.

10 Technique six: a length that is not a power of two

A network is built for a fixed number of wires, and every sorter in this note has a power of two of them. Real code has to sort eleven elements as well. There are two ways out, and djbsort uses both.

The first is **padding**. Fill the tail with a value above everything else, sort the lot, and drop the padding: the sort has pushed it to the end, so what is left in front is the answer. This is what the C does for a short array, and what `avx2_prog_pad` says.

The second is **splitting at the top bit**, which is what `int32_sort_short` does for a long one. Eleven is $8 + 2 + 1$. Sort the first eight downwards, sort the remaining three the same way — the routine calls itself — and the array now falls and then rises. That is exactly the shape a bitonic merge undoes, so one merge finishes the job. Peel the top bit again and again and the blocks of the recursion are simply the bits of n, from the highest down.

But a merge of eleven wires is not a network either. The code uses the merge of the next power of two, sixteen, and drops every comparison that reaches past place ten. Why that is sound is worth spelling out, because it is the one place where the padding idea reappears as an argument rather than as code. Imagine the five missing places filled with a value above everything. A comparison between two real places behaves as before. A comparison that reaches into the imaginary region compares a real value with the top value, puts the smaller — the real one — at the low place, and the top value at the high one, which is imaginary: nothing in the array has moved. So the pruned merge does to the eleven places exactly what the full merge would do to the padded sixteen, and the full merge sorts. That is `nfun_pnet_padt`, and the statement it gives is

```
Theorem sorted_hcr_prune (p n : nat) (s1 s2 : seq bool) (t : n.-tuple bool) :
  n <= `2^ p -> sorted >=%O s1 -> sorted <=%O s2 -> t = s1 ++ s2 :> seq bool ->
  sorted <=%O
    (nfun (pnet n [seq ab <- nstages (half_cleaner_rec false p) | ab.2 < n]) t).
```

which reads: an array that falls and then rises is sorted by the merge of the next power of two, with the comparisons that leave the array filtered out.

There is a trap in that argument. It only works when a comparison sends the smaller value to the **lower** place. Every comparison of the merge does. The comparisons of the block sorter do not — half of them run downwards, which is the mask trick of section 7 seen from the list side. So the two ways out are not interchangeable: the merge may be pruned, the blocks must be padded.

Sorting a block downwards, at the level of lists, is one line: turn every comparison round. Where the upward sorter compares (a, b) — smaller to a — the downward one compares (b, a). That this really sorts downwards is the same complement argument as in section 7, now in two lines: flipping every value turns one comparison round, and flipping an ascending list gives a descending one.

Put together, one recursive call is

```
Theorem sorting_smerge (p q m : nat) (l1 l2 : seq (nat * nat)) :
  q + m <= `2^ p ->
  all (fun ab => (ab.1 < q) && (ab.2 < q)) l1 ->
  pnet q l1 \is sorting -> pnet m l2 \is sorting ->
  pnet (q + m) (smerge p q m l1 l2) \is sorting.
```

— whatever sorts the first block and whatever sorts the rest, the merge on top of them sorts the whole — and the recursion is that lemma applied to the bits of n. Running it with djbsort's own AVX2 comparisons as the block sorter gives `sorting_avx2_short`, in `code/avx2/proof/sort_short.v`: a sorting network for every length.

One more change of view is needed before the code can be read against this. The merge is **written** as a recursion — one cleaner across the whole array, then two copies of itself on the halves — and no code runs it that way. Code runs one distance at a time, across the whole array: every comparison

at distance $\frac{n}{2}$, then every one at $\frac{n}{4}$, down to 1. The two descriptions are the same list, and `code/common/nlevel.v` proves it:

```
Theorem nstagesl_half_cleaner_rec (p : nat) :
  [seq cpairs c | c <- half_cleaner_rec false p]
    = [seq level_pairs (`2^ p) d d false | d <- dists p].
```

where `level_pairs N d d false` is one level: every pair $(i, i+d)$ with $i + d < N$ and the d-bit of i clear, in increasing i. Two small facts carry the induction: one cleaner **is** one level, and putting two copies of an array side by side doubles each level exactly (which needs $2d$ to divide the half-width — true here because every distance is a power of two). Pruning then just shortens each level, so the merge on an awkward length is `flatten [seq level_pairs n d d false | d <- dists p]`.

That list is what the loops of `code/avx2/c/sort_short.c` have to be matched against, and the first half of the match is settled. Look at what the code does at one distance:

```
long long j = stage(x,n,8,q,N_mrg8);
minmax_vector(&x[j], &x[j + 4*q], n - 4*q - j);
```

`stage` runs the whole blocks while they fit and returns where it stopped; `minmax_vector` then handles whatever ragged piece is left, with a length computed from n. A level has exactly that shape, and it is an equality of lists, not merely of sets — both sides list the comparisons by increasing low place:

```
Theorem level_pairs_blocks (n d : nat) : 0 < d ->
  level_pairs n d d false
    = flatten [seq mm (m * d.*2) (m * d.*2 + d) d | m <- iota 0 (n %/ d.*2)]
        ++ mm (n %/ d.*2 * d.*2) (n %/ d.*2 * d.*2 + d)
              (n - d - n %/ d.*2 * d.*2).
```

where `mm a b len` is what `minmax_vector(&x[a], &x[b], len)` compares. Eleven wires at distance two, for instance, are two whole blocks and then a tail of one comparison: $(0,2), (1,3), (4,6), (5,7)$, and $(8,10)$. The truncated subtraction does real work here: when fewer than d places are left the tail length comes out as zero, which is exactly what the C does when `n - 4*q - j` goes negative and `minmax_vector` returns without comparing anything.

The other half of the match — what the code does **inside** a block, where it works eight lanes at a time and fuses three distances into one pass — is the next section.

11 Technique seven: eight lanes at a time

A level at distance d compares place i with place $i + d$. The code never walks it that way.

One distance, eight at a time. It loads eight consecutive places into one register, the eight places q further on into another, and compares the two registers: one instruction, eight comparisons. Then it moves eight lanes on. The lane groups come in increasing order, so this is not even a reordering — the list the code emits is the level itself, comparison for comparison:

```
Theorem level_pairs_vstage (n q : nat) : 0 < q -> 8 %| q ->
  level_pairs n q q false
    = vstage n 2 q [:: (0, 1)]
      ++ mm (n %/ q.*2 * q.*2) (n %/ q.*2 * q.*2 + q)
            (n - q - n %/ q.*2 * q.*2).
```

Three distances at once. With eight registers in hand the code does not stop at one distance. `N_mrg8` is a table of twelve compare-exchanges **between registers**, and running it does the work of three levels while the values stay where they are. That is where the reordering is.

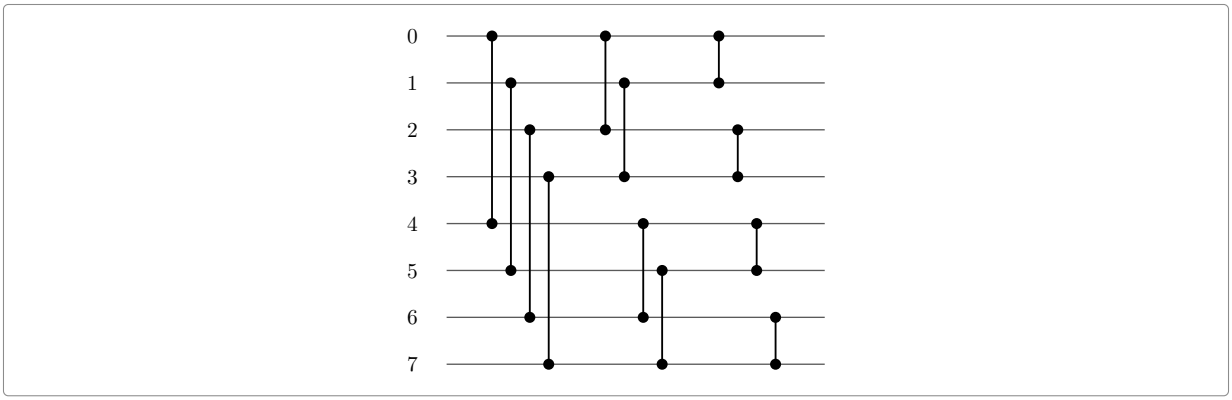

Figure 8: The table `N_mrg8`: twelve comparisons between eight registers. It is the bitonic merge on eight wires, one distance at a time. Each wire here is q consecutive places of the array, so each comparison drawn is q comparisons of the array, and the three columns are the levels at $4q$, at $2q$ and at q.

Two observations turn that picture into a proof.

The first: the tables **are** merges. The merge on eight wires, read one distance at a time, is `P_mrg8` of the C source, letter for letter — and the same holds at four registers and at two:

```
Example nstages_hcr_3 :
  nstages (half_cleaner_rec false 3)
    = [:: (0, 4); (1, 5); (2, 6); (3, 7); (0, 2); (1, 3); (4, 6); (5, 7);
          (0, 1); (2, 3); (4, 5); (6, 7)].
Proof. by rewrite nstages_half_cleaner_rec. Qed.
```

The second: a register stands for q consecutive places, and blowing every wire of a level up into q places gives a level again, at q times the distance — comparison by comparison and, inside each, lane by lane:

```
Theorem level_pairs_scale (N d q : nat) : 0 < d -> 0 < q ->
  level_pairs (N * q) (d * q) (d * q) false
    = flatten [seq mm (ab.1 * q) (ab.2 * q) q | ab <- level_pairs N d d false].
```

Together they say that the batch, run on a block of $8q$ places, is the levels at $4q$, $2q$ and q of that block. What is left to justify is the order inside the block: the code goes lane group by lane group, the levels go comparison by comparison. Colour a comparison by its lane group — for a place x, that is `x %% q %/ 8` — and section 6 settles it, for **any** table of comparisons between registers:

```
Theorem vblock_dequiv (n base q c : nat) (g : seq (nat * nat)) :
  0 < q -> 8 %| q -> q %| base -> base + c * q <= n ->
  all (fun ab => (ab.1 < c) && (ab.2 < c)) g ->
  dequiv n (vblock base q q g)
           (flatten [seq mm (base + ab.1 * q) (base + ab.2 * q) q | ab <- g]).
```

One turn of the loop. Now the loop itself can be read. It takes three distances at a time:

```
while (q >= 64) {
  q >>= 2;
  long long j = stage(x,n,8,q,N_mrg8);
  minmax_vector(&x[j], &x[j + 4*q], n - 4*q - j);
  if (j + 4*q <= n) { blockn(x,j,q,q,4,N_mrg4); j += 4*q; }
  minmax_vector(&x[j], &x[j + 2*q], n - 2*q - j);
  if (j + 2*q <= n) { blockn(x,j,q,q,2,N_mrg2); j += 2*q; }
  minmax_vector(&x[j], &x[j + q], n - q - j);
  q >>= 1;
}
```

The stage runs the whole blocks of $8q$ places, and each of them fuses the three levels. The six lines after it pick up what the stage could not reach: the level at $2q$ may have one whole block more than the stage ran, the level at q up to three, and all three levels have a ragged tail. The counts are arithmetic on what n leaves over $8q$: writing r for that remainder, the level at $2q$ has $2(n \div 8q) + r \div 4q$ whole blocks and the level at q has $4(n \div 8q) + r \div 2q$, and the leftover blocks the code runs are exactly the extra ones.

Then the order, and here the picture is simple. Everything the stage does lies below the place j it stopped at; everything after it starts at j or later. Two comparisons on opposite sides of j share no place, so three moves of whole blocks — each past a block it shares no place with — put the turn into level order: first all of the level at $4q$, then all of the level at $2q$, then all of the level at q.

One detail refuses to fit that frame. When the length handed to `minmax_vector` is not a multiple of eight, the routine does the **last** eight first and then walks the whole eights from the start, so eight of its comparisons are performed twice. A reordering cannot account for that: two lists that differ by a repeat are not rearrangements of one another. What is true is that they compute the same function, because a comparison done twice is the comparison:

```
Definition nequiv (n : nat) (l1 l2 : seq (nat * nat)) : Prop :=
  forall (d : disp_t) (A : orderType d) (t : n.-tuple A),
    nfun (pnet n l1) t = nfun (pnet n l2) t.

Lemma nequiv_dup (n a b : nat) (l : seq (nat * nat)) : a < n -> b < n ->
  nequiv n ((a, b) :: (a, b) :: l) ((a, b) :: l).
```

Every reordering is an `nequiv`, and so is dropping a repeat. With those, one turn of the loop is three levels of the merge:

```
Theorem mbody_levels (n q : nat) : 0 < q -> 8 %| q ->
  nequiv n (mbody n q)
           (level_pairs n (4 * q) (4 * q) false
            ++ level_pairs n (2 * q) (2 * q) false
            ++ level_pairs n q q false).
```

where `mbody n q` is the turn written out — the stage, the two leftover blocks and the three tails, in the order the C runs them.

11.1 All the turns

One turn is three levels; the loop is all of them. Look at what it does to q: it shifts right by two before the body and by one after, so each turn divides q by eight — three halvings, three levels. Nothing else about the loop matters. Writing 2^j for the body's q on the last turn, k turns are

```
Fixpoint mturns (n j k : nat) : seq (nat * nat) :=
  if k is k1.+1 then mbody n (`2^ (j + 3 * k1)) ++ mturns n j k1 else [::].
```

and one line of `mbody_levels` per turn gives the $3k$ levels they run, `dtop j k`, largest distance first. The turn stops where q would fall below eight lanes, so $j \geq 3$; that is the only condition.

What makes this the **whole** merge is one list identity: those $3k$ distances are the top ones of a merge on 2^{j+3k} wires, and what is left below them is a merge on 2^j wires.

```
Lemma dists_dtop (j k : nat) : dists (j + 3 * k) = dtop j k ++ dists j.
```

So the loop, followed by anything at all that runs the distances it stopped above, is the pruned merge of section 10 — and therefore sorts:

```
Theorem sorted_mturns (n j k : nat) (L : seq (nat * nat)) (s1 s2 : seq bool)
    (t : n.-tuple bool) :
  3 <= j -> n <= `2^ (j + 3 * k) ->
  nequiv n L (flatten [seq level_pairs n d d false | d <- dists j]) ->
```

```
  sorted >=%O s1 -> sorted <=%O s2 -> t = s1 ++ s2 :> seq bool ->
  sorted <=%O (nfun (pnet n (mturns n j k ++ L)) t).
```

Read the hypothesis on `L` as a contract with the rest of the code: whatever the last phase of `int32_sort_short` does, if it performs the levels at 2^{j-1} down to 1, the merge is complete.

A thousand elements make that concrete. The code enters the merge with $q = 512$, so the first turn runs the levels at 512, 256 and 128 and the second those at 64, 32 and 16; then q is 8, the test `q >= 64` fails, and the loop is over. Here $j = 4$ and $k = 2$: two turns, six levels, and $2^{4+6} = 1024$, which is the width whose merge covers a thousand places. The contract left for the last phase is the four levels at 8, 4, 2 and 1.

12 What is proved, and what is not

The four main statements are these.

statement	what it says
`sorted_avx2_prog`	running the model of the AVX2 program returns a sorted array
`sorting_int32_sort_network`	the comparison sequence of the portable code sorts, for every length
`isort_pbsort`	the loop nest of the generic AVX2 sort **is** the bitonic network
`sorting_avx2_short`	the AVX2 program, inside the recursion of section 10, sorts every length

Each of them is closed: asking Rocq `Print Assumptions` on any of them answers *closed under the global context*, which means no axiom and no unfinished proof is involved.

The statements do not cover the same ground, and it is worth being precise. The portable statement holds for every length. `sorted_avx2_prog` is about arrays whose length is a power of two and at least sixty-four, which is the case the vector code is written for; a length that is not a power of two is dealt with in two ways, both proved — padding the array with a value above everything else and dropping the padding afterwards, and the recursion of section 10, whose blocks are the bits of the length.

One gap is left inside that last statement. From sixty-four elements up, the blocks are sorted by djbsort's own comparisons. Below that, the C runs a straight line it keeps for eight, sixteen and thirty-two elements, which has not been modelled; a bitonic sort of the same width stands in for it, so at those three widths the theorem is about a network the C does not run.

What is **not** proved is the step from the C text to the model of it. In the portable track that gap is written down honestly, as the only assumption in the development:

```
Parameter sortc_trace : nat -> seq (nat * nat).
Axiom sortc_faithful : forall n, sortc_trace n = me_pairs n.
```

which says: the comparisons the C source really performs are the ones the proof works with. Closing it needs a semantics for C, which is a separate undertaking. In the meantime the transcription is checked by running both and comparing the traces, for many sizes, with the small OCaml programs in `code/avx2/ml/` and `code/portable4/ml/`.

The AVX2 track now says the same thing about itself, in `code/avx2/proof/sort_c.v`, and it is worth noticing what that assumption is careful **not** to say. It covers lengths that are a power of two and at least sixty-four — the range where the model really is a transcription of the code, instruction by instruction — and, through padding, everything that the code sorts by padding. It says nothing about the loops `code/avx2/c/sort_short.c` runs for a longer array of an awkward length. Their model is the scheme of section 10, which is mathematics rather than a transcription: that the code's merge loop performs that scheme's pruned merge is a proof, not an assumption, and writing it down as an axiom would hide the work instead of leaving it in plain sight. Section 11 does most of that proof:

the merge loop is exactly the levels it should perform, from the largest distance down to the point where it stops, and everything in it is closed like the statements above. What is left is the phase after the loop, where the distances have become small enough that the code merges whole registers with shuffles instead of sweeping the array — the `L` of `sorted_mturns` — and, at the other end, the straight lines the C keeps for eight, sixteen and thirty-two elements.

13 The files

file	lines	what is in it
`code/common/nsort.v`	804	networks, and what it means to sort
`code/common/nbitonic.v`	664	the bitonic sorter
`code/common/nalgebra.v`	1796	the algebra of comparisons: `dequiv`, `nequiv`, and their toolkit
`code/common/nprog.v`	751	programs: `Cmp`, `Vcmp`, `Vshuf`, and `pflat`
`code/common/nprune.v`	232	padding, and the merge with comparisons dropped
`code/common/nrec.v`	454	the recursion for a length that is not a power of two
`code/common/nlevel.v`	625	the merge one distance at a time, and eight lanes at a time
`code/common/nmloop.v`	872	the merge loop of the AVX2 code
`code/portable4/proof/nbjsort.v`	1291	Knuth's merge exchange
`code/portable4/proof/int32_knuth.v`	602	the portable code is that network
`code/avx2/proof/sort_generic.v`	592	the bitonic network, and the loop nest
`code/avx2/proof/sort_transpose.v`	722	the transpose realisation
`code/avx2/proof/sort_prog.v`	596	djbsort's AVX2 code, as a program
`code/avx2/proof/sort_link.v`	4426	the bridge for the AVX2 code
`code/avx2/proof/sort_short.v`	85	that code inside the recursion, at every length
`code/avx2/proof/sort_c.v`	86	what is still assumed about the C source

The bridge is by far the largest file, which is the honest summary of this note: proving that a network sorts is textbook work, and proving that a real program performs that network is where the effort goes.

14 What to take away

The recipe, in five steps.

1. **Make the program branch-free.** Then the comparisons it performs are fixed in advance, and the program is a network.
2. **Use zeros and ones.** Sorting is settled by the two-valued case, which is what makes induction on networks work at all.
3. **Write the program down in a small language,** and read off the comparisons it performs, renaming as the shuffles move values about.
4. **Prove that the order does not matter,** by colouring the comparisons so that different colours never touch the same place. And when the code does not merely reorder — when it performs a comparison twice, as `minmax_vector` does on a ragged length — ask for less: not the same list, only the same function.
5. **Deal with the tricks separately:** complementing values instead of sorting downwards, and a loop nest instead of a recursion. Each is a small theorem once it is stated in the right way.

The last step is the one that keeps its shape across very different programs. The AVX2 code and the portable code look nothing alike, and both come down to the same sentence: the program compares the same pairs as the network, in an order that costs nothing.